



Clases e Interfaces en Kotlin

Programación Orientada a Objetos en Kotlin

Herencia

Clases Abstractas

Interfaces

Delegación

Tour Intermedio de Kotlin

¿Por qué compartir código?

El Desafío

Al crear proyectos complejos, necesitas compartir código entre clases de forma **eficiente y segura**.

 Las jerarquías de clases permiten **reutilizar código** y mantener un **comportamiento consistente**.

Beneficios



Evitar Duplicación

Escribe una vez, reutiliza en múltiples clases



Mantenibilidad

Cambios centralizados en un solo lugar



Seguridad

Diseño que previene herencia accidental



Organización

Estructura clara y jerárquica del código

¿Qué es la Herencia?

Mecanismo que permite crear una nueva clase basada en una clase existente, compartiendo sus propiedades y funciones.



Clase Padre

Clase base de la que se hereda



Clase Hijo

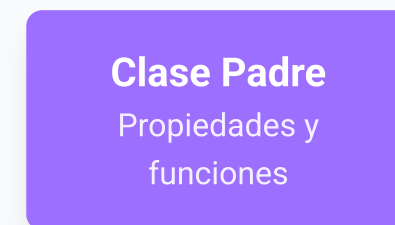
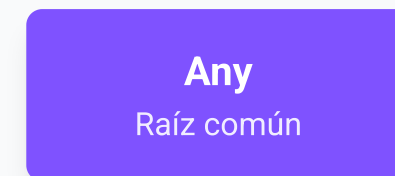
Clase que hereda de la clase padre



Jerarquía

Estructura organizada de clases relacionadas

Jerarquía de Clases



i La herencia promueve la **reutilización de código** y mantiene la **consistencia** en tu proyecto.

Clases Cerradas por Defecto

En Kotlin, las clases **no pueden heredarse por defecto** para prevenir herencia no intencional y facilitar el mantenimiento.

Herencia Simple

Solo se puede heredar de **una clase a la vez**

Diseño Seguro

Evita modificaciones accidentales del comportamiento base

Ventajas del Diseño



Previsión de Errores

Reduce errores causados por herencia inesperada



Mantenibilidad

Código más fácil de entender y mantener



Claridad

Intención explícita al permitir herencia



Para permitir herencia, usa la palabra clave **open**

🏠 Clase Any: La Raíz de la Jerarquía

Todas las Clases Heredan de Any

Any es la clase padre común de todas las clases en Kotlin. Proporciona métodos fundamentales heredados automáticamente.

Funciones Heredadas

Σ `toString()` Representación en texto

↔ `equals()` Comparación de objetos

`hashCode()` Código hash del objeto

Ejemplo Práctico

```
class Empleado(val nombre: String)
fun main() {
    val empleado = Empleado("María García")

    // Funciones heredadas de Any
    println(empleado.toString())
}
```

📄 **Salida:** Empleado@5e2de80c

Cada clase puede sobrescribir `toString()` para personalizar la salida

▲ Clases Abstractas

Definición y Propósito

Las clases abstractas están diseñadas para ser **heredadas**, proporcionando una estructura común que las clases hijas deben implementar.



Heredables por Defecto

No necesitan la palabra clave `open`



Constructor sin Instancias

Tienen constructor pero **no se pueden instanciar**



Miembros Mixtos

Funciones con y sin implementación

Sintaxis Básica

```
abstract class Vehiculo(  
    val marca: String,  
    val modelo: String  
) {  
    // Función con implementación  
    fun mostrarInfo() {  
        println("$marca $modelo")  
    }  
    // Función abstracta sin implementación  
    abstract fun arrancar()  
}
```

i Usa **abstract** para declarar la clase y sus miembros sin implementación

<> Sintaxis de Clases Abstractas

Clase Abstracta Padre

```
abstract class Empleado(  
    val nombre: String,  
    val salario: Double  
) {  
    // Propiedad abstracta  
    abstract val departamento: String  
    // Función con implementación  
    fun mostrarDatos() {  
        println("Nombre: $nombre")  
        println("Salario: €$salario")  
    }  
    // Función abstracta sin implementación  
    abstract fun calcularBonus(): Double  
}
```

Clase Hija

```
class Desarrollador(  
    nombre: String,  
    salario: Double,  
    val lenguaje: String  
) : Empleado(nombre, salario) {  
    // Sobrescribir propiedad abstracta  
    override val departamento = "Tecnología"  
    // Sobrescribir función abstracta  
    override fun calcularBonus(): Double {  
        return salario * 0.15  
    }  
}
```

💡 `override` es obligatorio para miembros abstractos

Clase Abstracta Libro

Definimos una **clase abstracta** que representa la estructura común de todos los libros de la biblioteca.



Propiedades Básicas

nombre y autor comunes a todos los libros



Tipo Abstracto

Cada subclase define su tipo específico



Método de Información

Función para mostrar datos del libro

Código de la Clase

```
abstract class Libro(  
    val nombre: String,  
    val autor: String  
) {  
    // Propiedad abstracta  
    abstract val tipo: String  
    // Función con implementación  
    fun mostrarInformacion() {  
        println("Libro: $nombre")  
        println("Autor: $autor")  
        println("Tipo: $tipo")  
    }  
}
```



La clase define la estructura común, las subclases implementarán el tipo específico

Implementación de Clase Hija

```
class LibroDigital(  
    nombre: String,  
    autor: String,  
    val formato: String, // PDF, EPUB, etc.  
    val tamañoMB: Double  
) : Libro(nombre, autor) {  
    // Sobrescribir propiedad abstracta  
    override val tipo = "Libro Digital"  
    fun descargar() {  
        println("Descargando $nombre...")  
        println("Tamaño: $tamañoMB MB")  
    }  
}
```

Características Clave



Herencia de Libro

Hereda nombre y autor de la clase padre



override val tipo

Define el tipo como "Libro Digital"



Propiedades Adicionales

formato y tamañoMB específicos



Función Propia

descargar() específica para digitales



Cada tipo de libro implementa su propio comportamiento

Implementación de Clase Hija

```
class LibroFisico(  
    nombre: String,  
    autor: String,  
    val paginas: Int,  
    val isbn: String  
) : Libro(nombre, autor) {  
    // Sobrescribir propiedad abstracta  
    override val tipo = "Libro Físico"  
    fun prestar(usuario: String) {  
        println("Libro '$nombre' prestado a $usuario")  
        println("ISBN: $isbn")  
    }  
}
```

Características Clave



Herencia de Libro

Hereda nombre y autor de la clase padre



override val tipo

Define el tipo como "Libro Físico"



Propiedades Adicionales

paginas e isbn específicos



Función Propia

prestar() específica para físicos



Cada tipo de libro puede tener comportamiento único

📌 Interfaces: Definición y Características

¿Qué es una Interface?

Las interfaces definen un **contrato de comportamiento** que las clases deben implementar.

Características Principales



No se pueden instanciar

No crean objetos directamente



Sin constructor

No requieren inicialización



Miembros implícitamente abiertos

No necesitan palabra clave open



Sin abstract obligatorio

Los miembros sin código son abstractos automáticamente

Sintaxis Básica

```
interface Reproducible {  
    // Propiedad sin implementación  
    val titulo: String  
    // Función sin implementación  
    fun reproducir()  
    // Función con implementación  
    fun pausar() {  
        println("Reproducción pausada")  
    }  
}
```



Las interfaces permiten **herencia múltiple** en Kotlin

↔ Interfaces vs Clases Abstractas

Tabla Comparativa

Característica	Interface	Clase Abstracta
⊘ Instanciable	✗	✗
🔧 Constructor	✗	✓
📐 Herencia Múltiple	✓	✗
<> Miembros con Código	✓	✓
☰ Estado (propiedades con backing field)	✗	✓

Cuándo Usar Cada Uno



Interface

- > Cuando necesites **herencia múltiple**
- > Para definir **contratos de comportamiento**
- > Tipos de comportamiento **independientes**



Clase Abstracta

- > Cuando necesites **compartir estado**
- > Para **jerarquías de clases** fuertes
- > Cuando requieras **constructor**

<> Declaración de Interfaces: Sintaxis Básica

Ejemplo: Interface MetodoPago

```
interface MetodoPago {  
    // Propiedad sin implementación  
    val nombreMetodo: String  
    // Función sin implementación  
    fun procesarPago(monto: Double): Boolean  
    // Función con implementación por defecto  
    fun obtenerDetalles() {  
        println("Método: $nombreMetodo")  
    }  
    // Otra función sin implementación  
    fun validarDatos(): Boolean  
}
```

💡 **interface** define el contrato que las clases deben cumplir

Elementos de Sintaxis

📌 Declaración

Usar palabra clave **interface**

Propiedades

Pueden ser abstractas o con implementación

Σ Funciones

Sin cuerpo: obligatorias. Con cuerpo: opcionales

👁 Modificadores

Sin **abstract**, **open** es implícito

Sistema de Notificaciones: Interface Simple

Interface Notificacion

```
interface Notificacion {  
    // Propiedad abstracta: tipo de notificación  
    val tipo: String  
    // Función para enviar la notificación  
    fun enviar(destinatario: String, mensaje: String)  
    // Función para obtener el estado del envío  
    fun obtenerEstado(): String  
    // Función para verificar si el envío fue exitoso  
    fun fueExitoso(): Boolean  
}
```

 Interface define **qué hace**, no **cómo lo hace**

Contexto de Aplicación

Notificaciones por Email

Envío de correos electrónicos

Notificaciones SMS

Mensajes de texto

Push Notifications

Alertas móviles en tiempo real

Mensajería Interna

Notificaciones dentro de la app

 **Polimorfismo:** Mismo contrato, diferentes implementaciones

✉ Implementación: EmailNotificacion

Clase EmailNotificacion

```
class EmailNotificacion(val remitente: String) : Notificacion {
    private var estadoEnvio = "Pendiente"
    private var exito = false
    // Sobrescribir propiedad de la interface
    override val tipo: String = "Email"
    override fun enviar(destinatario: String, mensaje: String) {
        println("📧 Enviando email de: $remitente")
        println("    Para: $destinatario")
        println("    Mensaje: $mensaje")
        estadoEnvio = "Enviado"
        exito = true
    }
    override fun obtenerEstado(): String = estadoEnvio
    override fun fueExitoso(): Boolean = exito
}
```

i `override` es obligatorio para implementar miembros de la interface

Puntos Clave

🔗 Implementación de Interface

Usar : `Notificacion` para implementar

↔ Estado Interno

Propiedades privadas para mantener estado

🔧 Lógica Propia

Implementación específica para emails

🧩 Sin Paréntesis

Interfaces no tienen constructor

▶ Uso:

```
val notif =
    EmailNotificacion("soporte@app.com")
notif.enviar("usuario@email.es",
    "Bienvenido")
```

Implementación: SMSNotificacion

Clase SMSNotificacion

```
class SMSNotificacion(val proveedor: String) : Notificacion {  
    private var estadoEnvio = "Pendiente"  
    private var exito = false  
    // Sobrescribir propiedad de la interface  
    override val tipo: String = "SMS"  
    override fun enviar(destinatario: String, mensaje: String) {  
        println("📠 Enviando SMS vía: $proveedor")  
        println("    Número: $destinatario")  
        // Limitar a 160 caracteres  
        val mensajeCorto = mensaje.take(160)  
        println("    Mensaje: $mensajeCorto")  
        estadoEnvio = "Entregado"  
        exito = true  
    }  
    override fun obtenerEstado(): String = estadoEnvio  
    override fun fueExitoso(): Boolean = exito  
}
```

 Cada implementación tiene su propia lógica de negocio

Diferencias con Email

Longitud Limitada

Mensajes de **160 caracteres máximo**

Proveedor SMS

Integración con **servicio externo**

Formato Específico

Número de teléfono como destinatario

Mismo Contrato, Diferente Implementación

Ambas clases cumplen la interface Notificacion pero con comportamientos distintos

Ejemplo:

```
val sms = SMSNotificacion("Twilio")  
sms.enviar("+34612345678", "Tu código es  
1234")
```


📌 Múltiples Interfaces: Concepto y Ventajas

Concepto Fundamental

Una clase puede implementar **múltiples interfaces** simultáneamente usando coma (,) para separarlas.

Ventajas Principales



Combinación de Comportamientos

Integrar funcionalidades de diferentes fuentes en una sola clase



Flexibilidad Arquitectónica

Diseño modular sin limitación de herencia simple



Reutilización de Código

Implementar contratos comunes en múltiples clases

Sintaxis y Ejemplo

<> Sintaxis Básica

```
class MiClase : InterfaceA, InterfaceB, etc.
```

```
// Definir múltiples interfaces
interface Rastreable {
    fun obtenerUbicacion(): String
}
interface Conectable {
    fun conectar(): Boolean
}
// Implementar ambas interfaces
class VehiculoConectado : Rastreable, Conectable {
    override fun obtenerUbicacion() = "Madrid, España"
    override fun conectar() = true
}
```

i Las interfaces se separan con **coma (,)** sin paréntesis

Sistema de Vehículos: Múltiples Interfaces

Definición de Interfaces

Interface Electrico

```
interface Electrico {  
    val bateria: Int  
    fun cargar()  
    fun nivelBateria(): Int  
}
```

Interface ConGPS

```
interface ConGPS {  
    fun obtenerUbicacion(): String  
    fun establecerDestino(destino: String)  
}
```

 Cada interface define un **comportamiento específico**

Implementación Múltiple

Clase CocheElectrico

```
class CocheElectrico(val modelo: String) : Electrico, ConGPS {  
    override var bateria = 0  
    private var ubicacion = "Desconocida"  
    private var destino = ""  
    override fun cargar() {  
        bateria = 100  
    }  
    override fun nivelBateria() = bateria  
    override fun obtenerUbicacion() = ubicacion  
    override fun establecerDestino(dest: String) {  
        destino = dest  
    }  
}
```

► Ejemplo de uso:

```
val tesla = CocheElectrico("Tesla Model 3")  
tesla.cargar()  
tesla.establecerDestino("Barcelona")
```

★ Beneficios de Usar Interfaces

Ventajas Principales

🏗️ Modularidad

Aísla componentes permitiendo que evolucionen de forma independiente

🐛 Testabilidad

Facilita pruebas unitarias intercambiando implementaciones reales por mocks

🔧 Mantenibilidad

Código más claro agrupando funciones relacionadas en conjuntos cohesivos

Ejemplo Práctico

<> Sistema de Pagos Modular

```
interface MetodoPago {  
    fun procesar(monto: Double): Boolean  
}  
  
// Implementación real  
class PagoConTarjeta : MetodoPago {  
    override fun procesar(monto: Double) = true  
}  
  
// Mock para pruebas  
class PagoMock : MetodoPago {  
    override fun procesar(monto: Double) = false  
}
```

✓ Contrato claro

✓ Flexibilidad

✓ Reutilización

✓ Escalabilidad

⚠ Delegación: El Problema del Código Repetitivo

El Problema



Boilerplate Code

Interfaces con muchas funciones generan **código repetitivo** en cada implementación

🔗 ¿Qué ocurre?

- ▶ Interface con **10+ funciones**
- ▶ Solo necesitas cambiar **1 función**
- ▶ Debes reimplementar **las 10 funciones**

🔴 **Impacto negativo:** Duplicación, errores, difícil mantenimiento

Ejemplo del Problema

📄 Interface con muchas funciones

```
interface HerramientaDibujo {  
    val color: String  
    fun dibujarLinea()  
    fun dibujarRectangulo()  
    fun dibujarCirculo()  
    fun seleccionar()  
}
```

📄 Implementación con mucho código

```
class SesionCanvas(val herramienta: HerramientaDibujo) : HerramientaDibujo {  
    override val color = "Azul" // Solo cambio esto  
  
    // Debo reimplementar todas las funciones  
    override fun dibujarLinea() { herramienta.dibujarLinea() }  
    override fun dibujarRectangulo() { herramienta.dibujarRectangulo() }  
    override fun dibujarCirculo() { herramienta.dibujarCirculo() }  
    override fun seleccionar() { herramienta.seleccionar() }  
}
```

✂️ Delegación con 'by': Sintaxis y Concepto

La Solución en Kotlin

✅ Delegación Automática

El compilador implementa automáticamente las funciones de la interface

★ Sintaxis con 'by'

```
class MiClase(delegado: MiInterface) : MiInterface by delegado
```

Ventajas

- ✓ Sin código repetitivo
- ✓ Solo sobrescribes lo necesario
- ✓ Código más limpio y mantenible

Ejemplo con Delegación

📄 Interface HerramientaDibujo

```
interface HerramientaDibujo {  
    val color: String  
    fun dibujarLinea()  
    fun dibujarRectangulo()  
    fun dibujarCirculo()  
    fun seleccionar()  
}
```

✅ Sin código repetitivo

```
class SesionCanvas(  
    private val herramienta: HerramientaDibujo  
) : HerramientaDibujo by herramienta {  
    // Solo sobrescribo lo que necesito cambiar  
    override val color = "Azul"  
}
```

❗ Sistema de Reportes: Sin Delegación

Interface con Múltiples Funciones

📄 GeneradorReportes

```
interface GeneradorReportes {  
    fun generarCabecera(): String  
    fun generarCuerpo(datos: String): String  
    fun generarPie(): String  
    fun formatearFecha(fecha: String): String  
    fun exportar(contenido: String): ByteArray  
}
```

⚠️ 5 funciones para reimplementar

Clase Sin Delegación - Mucho Código

📄 ReportePersonalizado - Código Repetitivo

```
class ReportePersonalizado(private val reporteBase: GeneradorReportes) : GeneradorReportes {  
    // Solo queremos cambiar el formato de fecha  
    override fun formatearFecha(fecha: String): String {  
        return "Fecha: $fecha" // Cambio real  
    }  
    // Pero debemos reimplementar TODO lo demás  
    override fun generarCabecera() = reporteBase.generarCabecera()  
    override fun generarCuerpo(datos: String) = reporteBase.generarCuerpo(datos)  
    override fun generarPie() = reporteBase.generarPie()  
    override fun exportar(contenido: String) = reporteBase.exportar(contenido)  
}
```

✗ Duplicación

Mismo código en múltiples clases

✗ Propenso a errores

Cambios manuales en cada lugar

✓ Sistema de Reportes: Con Delegación

Comparación

✗ Sin Delegación

- 20+ líneas de código
- Código repetitivo
- Difícil mantenimiento

✓ Con Delegación

- 6 líneas de código
- Código limpio
- Fácil mantenimiento

Reducción de código

70%

Solución con Delegación - Código Limpio

✂ ReportePersonalizado - Delegación Automática

```
class ReportePersonalizado(  
    private val reporteBase: GeneradorReportes  
) : GeneradorReportes by reporteBase {  
    // Solo sobrescribo lo que necesito cambiar  
    override fun formatearFecha(fecha: String): String {  
        return "Fecha: $fecha" // Implementación personalizada  
    }  
}
```

🚀 Rápido

Menos código = más rápido

💡 Claro

Intención obvia

♥ Mantenible

Fácil de modificar

i Las otras 4 funciones se delegan automáticamente a reporteBase

★ Ventajas de la Delegación

Beneficios Clave



Menos Boilerplate

Reduce código repetitivo drásticamente



Control Preciso

Solo sobrescribes lo que necesitas cambiar



Automatización

El compilador hace el trabajo por ti



Código Limpio

Más legible y fácil de mantener

Comparación de Código

✗ Sin Delegación (20+ líneas)

```
class MiClase(delegado: Interface) : Interface {  
    override fun funcion1() = delegado.funcion1()  
    override fun funcion2() = delegado.funcion2()  
    // ... 10 funciones más  
}
```

✓ Con Delegación (3 líneas)

```
class MiClase(private val delegado: Interface) :  
    Interface by delegado {  
    // Solo funciones que necesites cambiar  
}
```

Reducción

85%

Menos errores



Más rápido





Conclusiones y Próximos Pasos

Resumen de Conceptos



Clases Abstractas

Para compartir **estado y comportamiento** común



Interfaces

Para definir **contratos y herencia múltiple**



Delegación

Para **reducir código repetitivo**



Combina estas herramientas para crear arquitecturas flexibles y mantenibles

Buenas Prácticas

- ✓ Prefiere **composición sobre herencia**
- ✓ Usa interfaces para **definir comportamientos**
- ✓ Aplica delegación cuando **sobrescribas pocas funciones**
- ✓ Mantén interfaces **pequeñas y cohesivas**
- ✓ Documenta el propósito de cada **clase abstracta**

→ Próximos Pasos

- Práctica con ejemplos del mundo real
- Explora clases selladas (sealed classes)
- Aprende sobre genéricos con interfaces